Tutorial 6 - Creating new features, ratios, binning, one-hot encoding, standardization, and normalization.

How to use this tutorial

- Run each example as a separate notebook cell or Python script.

1. Convert sales amounts from messy strings to numeric

Real-world context: A retail CSV stores sales values such as '$1,200', '$850', and 'N/A'. We want a clean numeric column for analysis.

```
import pandas as pd

df = pd.DataFrame({
    "product": ["Keyboard", "Mouse", "Monitor", "Chair"],
    "sales_raw": ["$1,200", "$850", "N/A", "$2,300"]
})

df["sales"] = pd.to_numeric(
    df["sales_raw"].str.replace("$", "", regex=False).str.replace(",", "",
regex=False),
    errors="coerce"
)

print(df)
print("\nTotal sales:", df["sales"].sum())
```

Question: What is observation?

After running the script, it tells me the total money earned from sales of products such as the keyboard, mouse, monitor and chair, being $4350.

2. Convert student age from text to integer-friendly numeric values

Real-world context: A student survey stores ages as text, with one invalid value. We need numeric ages before computing averages.

```
import pandas as pd

df = pd.DataFrame({
    "student": ["Asha", "Ben", "Chia", "Dina"],
    "age_raw": ["18", "19", "twenty", "20"]
})

df["age"] = pd.to_numeric(df["age_raw"], errors="coerce")

print(df)
print("\nAverage age:", df["age"].mean())
```

Question: What is observation? After executing the code, it shows the average age amongst the dataframe, the code printed out 19.0. It didnt take into account the non-numeric age in the dataframe

3. Convert percentage strings into decimal ratios

Real-world context: A marketing dashboard stores click-through rates as '12%', '8%', and '15%'. We want decimals for modelling.

```
import pandas as pd

df = pd.DataFrame({
    "campaign": ["Email A", "Email B", "Email C"],
    "ctr_raw": ["12%", "8%", "15%"]
})

df["ctr"] = pd.to_numeric(df["ctr_raw"].str.replace("%", "", regex=False)) / 100

print(df)
```

Question: What is observation?

After executing the code, the outputs were the same as the percentages in "ctr_raw", just that its '%' was removed and it was in decimal form.

4. Parse transaction dates into datetime

Real-world context: An e-commerce file stores order dates as text. We need proper datetime values to filter by month or weekday.

```
import pandas as pd

df = pd.DataFrame({
    "order_id": [101, 102, 103, 104],
    "order_date_raw": ["2026-01-05", "2026-01-09", "2026-02-01", "bad-date"]
})

df["order_date"] = pd.to_datetime(df["order_date_raw"], errors="coerce")

print(df)
print("\nMonth values:")
print(df["order_date"].dt.month)
```

Question: What is observation?

5. Extract day name and weekend flag from dates

Real-world context: A food delivery company wants to know whether orders arrive on weekends.

```
import pandas as pd

df = pd.DataFrame({
    "delivery_id": [1, 2, 3, 4],
    "delivery_date": pd.to_datetime(["2026-03-13", "2026-03-14", "2026-03-15", "2026-
03-16"])
})

df["day_name"] = df["delivery_date"].dt.day_name()
df["is_weekend"] = df["delivery_date"].dt.dayofweek >= 5

print(df)
```

Question: What is observation?

6. Calculate delivery duration in hours

Real-world context: A logistics company stores pickup and drop-off timestamps. We want delivery duration as a numeric feature.

```
import pandas as pd

df = pd.DataFrame({
    "pickup_time": pd.to_datetime(["2026-03-10 08:00", "2026-03-10 09:30", "2026-03-10 11:00"]),
    "dropoff_time": pd.to_datetime(["2026-03-10 10:00", "2026-03-10 12:00", "2026-03-10 12:15"])
})

df["duration_hours"] = (df["dropoff_time"] - df["pickup_time"]).dt.total_seconds() / 3600

print(df)
```

Question: What is observation?

7. Standardize inconsistent text labels

Real-world context: Customer records store city names in mixed case and with extra spaces. We want neat text for grouping.

```
import pandas as pd

df = pd.DataFrame({
    "customer_id": [1, 2, 3, 4],
    "city_raw": [" singapore", "SINGAPORE", "Jurong East ", "jurong east"]
})

df["city_clean"] = df["city_raw"].str.strip().str.title()

print(df)
```

Question: What is observation?

8. Create email domain as a new feature

Real-world context: A subscription platform wants to study sign-ups by email provider.

```
import pandas as pd

df = pd.DataFrame({
    "user": ["alice", "ben", "carol", "dan"],
    "email": ["alice@gmail.com", "ben@yahoo.com", "carol@gmail.com", "dan@school.edu"]
})

df["domain"] = df["email"].str.split("@").str[1]

print(df)
```

Question: What is observation?

9. Build profit and profit margin features

Real-world context: A store wants both total profit and profit margin from sales and cost columns.

```
import pandas as pd

df = pd.DataFrame({
    "item": ["A", "B", "C"],
    "sales": [1200, 800, 1500],
    "cost": [700, 500, 900]
})

df["profit"] = df["sales"] - df["cost"]
df["profit_margin"] = df["profit"] / df["sales"]

print(df)
```

Question: What is observation?

10. Create student attendance ratio

Real-world context: A school system stores lessons attended and total lessons. We want a ratio for performance analysis.

```
import pandas as pd

df = pd.DataFrame({
    "student": ["Ali", "Bea", "Chen", "Deepa"],
    "attended": [18, 20, 15, 19],
    "total_lessons": [20, 20, 20, 20]
})

df["attendance_ratio"] = df["attended"] / df["total_lessons"]

print(df)
```

Question: What is observation?

11. Bin customer ages into age groups

Real-world context: A marketing team wants age bands rather than raw ages.

```
import pandas as pd

df = pd.DataFrame({
    "customer": ["A", "B", "C", "D", "E"],
    "age": [17, 22, 35, 47, 61]
})

bins = [0, 18, 30, 45, 100]
labels = ["Teen", "Young Adult", "Adult", "Senior"]

df["age_group"] = pd.cut(df["age"], bins=bins, labels=labels, right=False)

print(df)
```

Question: What is observation?

12. Bin exam marks into performance bands

Real-world context: A lecturer wants to classify raw marks into Fail, Pass, Merit, and Distinction.

```
import pandas as pd

df = pd.DataFrame({
    "student": ["A", "B", "C", "D", "E"],
    "score": [42, 58, 67, 79, 91]
})

bins = [0, 50, 65, 80, 101]
labels = ["Fail", "Pass", "Merit", "Distinction"]

df["band"] = pd.cut(df["score"], bins=bins, labels=labels, right=False)

print(df)
```

Question: What is observation?

13. One-hot encode department names

Real-world context: An HR dataset contains department labels. Many ML models need numeric input.

```
import pandas as pd

df = pd.DataFrame({
    "employee": ["E1", "E2", "E3", "E4"],
    "department": ["Sales", "IT", "HR", "IT"]
})

encoded = pd.get_dummies(df, columns=["department"])
print(encoded)
```

Question: What is observation?

14. One-hot encode room type for hotel bookings

Real-world context: A hotel wants to prepare booking data for prediction models.

```
import pandas as pd

df = pd.DataFrame({
    "booking_id": [1, 2, 3, 4],
    "room_type": ["Standard", "Deluxe", "Suite", "Standard"],
    "nights": [2, 3, 1, 4]
})

df_encoded = pd.get_dummies(df, columns=["room_type"], dtype=int)

print(df_encoded)
```

Question: What is observation?

15. Standardize monthly income using z-score

Real-world context: Income values have a large range. We want values centered around 0 with standard deviation near 1.

```
import pandas as pd

df = pd.DataFrame({
    "employee": ["A", "B", "C", "D"],
    "income": [3000, 4500, 5000, 7000]
})

mean_income = df["income"].mean()
std_income = df["income"].std()

df["income_standardized"] = (df["income"] - mean_income) / std_income

print(df.round(3))
```

Question: What is observation?

16. Normalize sensor readings to the 0–1 range

Real-world context: An IoT system records temperature from different machines. We want all readings on a common scale.

```
import pandas as pd

df = pd.DataFrame({
    "machine": ["M1", "M2", "M3", "M4"],
    "temperature": [45, 55, 60, 80]
})

min_temp = df["temperature"].min()
max_temp = df["temperature"].max()

df["temperature_normalized"] = (df["temperature"] - min_temp) / (max_temp - min_temp)

print(df.round(3))
```

Question: What is observation?

17. Normalize product ratings to a 0–100 score

Real-world context: An online platform stores ratings on a 1–5 scale but wants to display a 0–100 score.

```
import pandas as pd

df = pd.DataFrame({
    "product": ["A", "B", "C", "D"],
    "rating": [2.5, 3.8, 4.2, 5.0]
})

df["rating_score"] = (df["rating"] - 1) / (5 - 1) * 100

print(df.round(1))
```

Question: What is observation?

18. Convert mixed yes/no text into numeric binary features

Real-world context: A telecom dataset stores churn survey answers as text. We want model-friendly 0/1 values.

```
import pandas as pd

df = pd.DataFrame({
    "customer": ["C1", "C2", "C3", "C4"],
    "auto_pay": ["Yes", "No", "Yes", "No"]
})

df["auto_pay_flag"] = df["auto_pay"].map({"Yes": 1, "No": 0})

print(df)
```

Question: What is observation?

19. Create revenue per employee ratio

Real-world context: A business analyst wants to compare branches of different sizes using a ratio instead of raw totals.

```
import pandas as pd

df = pd.DataFrame({
    "branch": ["North", "South", "East", "West"],
    "revenue": [120000, 98000, 143000, 110000],
    "employees": [12, 10, 14, 11]
})

df["revenue_per_employee"] = df["revenue"] / df["employees"]

print(df.round(2))
```

Question: What is observation?

20. Build a mini preprocessing pipeline in Pandas

Real-world context: This example combines several realistic steps: numeric cleaning, datetime parsing, feature creation, and encoding.

```
import pandas as pd

df = pd.DataFrame({
    "order_value_raw": ["$120", "$250", "bad", "$180"],
    "order_date_raw": ["2026-01-05", "2026-01-07", "2026-01-10", "2026-01-12"],
    "channel": ["Online", "Retail", "Online", "Partner"]
})

df["order_value"] = pd.to_numeric(
    df["order_value_raw"].str.replace("$", "", regex=False),
    errors="coerce"
)
df["order_date"] = pd.to_datetime(df["order_date_raw"], errors="coerce")
df["order_month"] = df["order_date"].dt.month
df["high_value"] = (df["order_value"] >= 200).astype(int)

df = pd.get_dummies(df, columns=["channel"], dtype=int)

print(df)
```

Question: What is observation?